Множества, JOIN и NULL в PostgreSQL

Репозиторий с докладами

Владимир Добрышкин

t.me/vodobryshkin

Версия 0.1.1

Аннотация

Данный документ представляет собой практическую памятку по:

- Множествам и мультимножествам.
- Соединениям таблиц.
- NULL и трёхзначной логике.

Содержание

База данных для примеров . . . 3
Предметная область . . . 3
Запуск базы данных . . . 4
Исходные данные . . . 4
NULL . . . 5
Трёхзначная логика . . . 5
Множества и мультимножества . . . 7
Определения . . . 7
Операции над множествами . . . 8
JOIN . . . 11
Визуализация JOIN . . . 11
CROSS JOIN – перекрёстное соединение . . . 12
INNER JOIN – внутреннее соединение . . . 13
LEFT JOIN – левое внешнее соединение . . . 14
RIGHT JOIN – правое внешнее соединение . . . 15
FULL JOIN – полное внешнее соединение . . . 16
NATURAL JOIN – естественное соединение . . . 17
Обратная связь . . . 18
Источники . . . 18

База данных для примеров

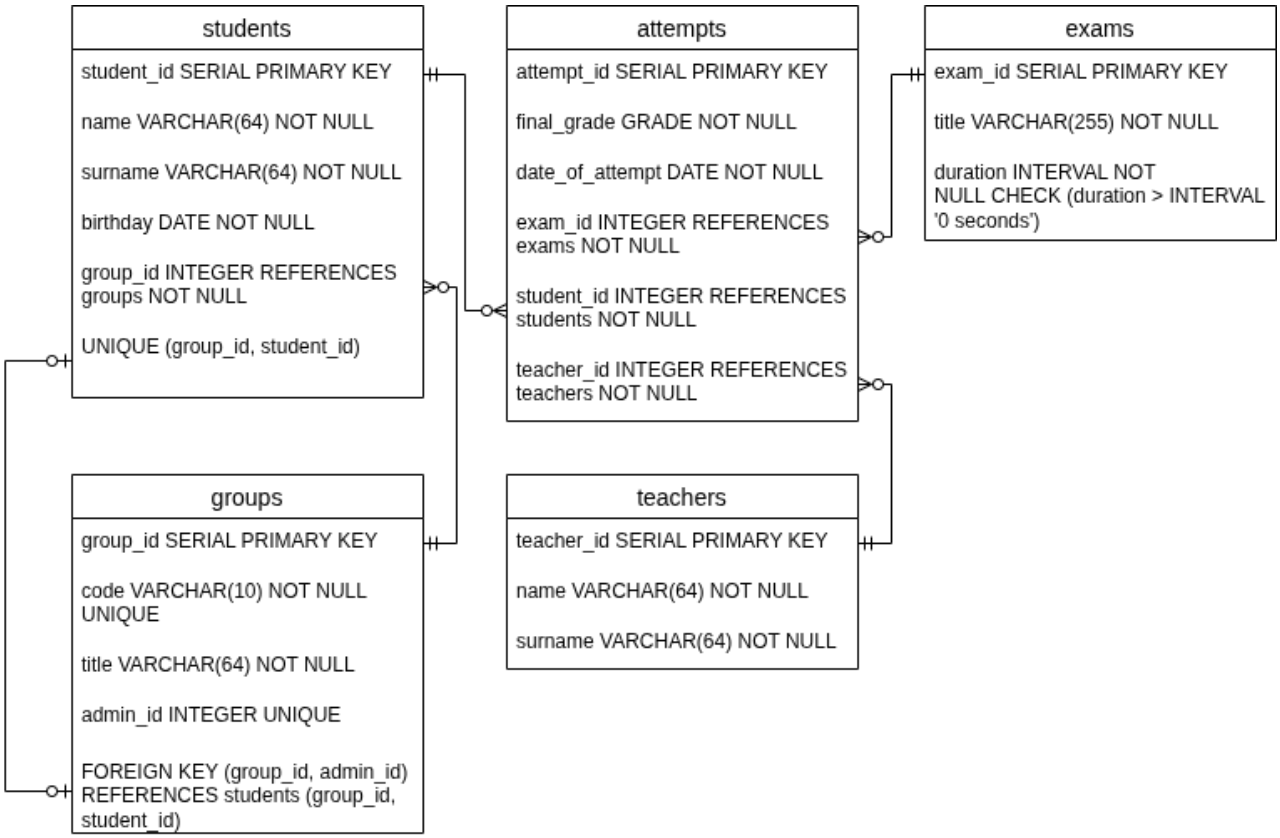
В этой памятке мы рассмотрим дополнительный материал к лабораторной работе № 2, который поможет глубже разобраться в SQL и соединениях таблиц.

> Тема практическая, поэтому я решил разобрать все примеры на одной учебной базе данных и приложить файлы для её запуска. При желании вы сможете самостоятельно повторить все запросы из памятки.

Все примеры используют одну учебную базу данных о сдаче экзаменов. Эта предметная область уже рассматривалась в памятке по оформлению моделей и SQL-кода.

Предметная область

- Студенты сдают экзамены преподавателям. Каждая попытка сдачи связывает студента, экзамен и преподавателя, а также содержит дату и оценку. Повторная сдача того же экзамена сохраняется как отдельная попытка.
- Каждый студент состоит ровно в одной учебной группе. Группа может быть пустой, для неё можно указать старосту из числа её студентов.
- У группы может быть один староста, выбранный из числа студентов этой же группы. Назначать старосту необязательно.



Изображение 1: Физическая модель базы данных

Запуск базы данных

Для запуска понадобится Docker.

1. Скачайте файлы `compose.yaml` и `init.sql` из репозитория и сохраните их в **одну директорию**.
2. Откройте терминал в этой директории и запустите БД:

```
docker compose up -d --wait
```

3. После завершения запуска подключитесь к БД через `psql`:

```
docker compose exec db psql -U report -d exams-db
```

Исходные данные

Содержимое таблиц после выполнения `init.sql`. Внешние ключи содержат идентификаторы соответствующих записей.

students

student_id	name	surname	birthday	group_id
1	Иван	Иванов	2006-01-01	1
2	Петр	Петров	2006-02-01	1
3	Федор	Федоров	2006-03-01	1
4	Дарья	Алексеева	2006-04-01	2
5	Елена	Волкова	2006-05-01	2
6	Егор	Морозов	2006-06-01	2
7	Анна	Смирнова	2006-07-01	1
8	Дмитрий	Дмитриев	2006-08-01	2

groups

group_id	code	title	admin_id
1	P3221	Первая группа	1
2	P3222	Вторая группа	4
3	P3223	Третья группа	NULL

teachers

teacher_id	name	surname
1	Ольга	Лебедева
2	Павел	Зайцев
3	Мария	Новикова

exams

exam_id	title	duration
1	Базы данных	01:30:00
2	Мат. анализ	02:00:00
3	Програм-мирование	01:30:00
4	Операционные системы	01:00:00

attempts

attempt_id	student_id	exam_id	teacher_id	date_of_attempt	final_grade
1	1	1	1	2026-09-01	2
2	1	1	1	2026-09-08	4
3	1	1	1	2026-09-15	5
4	1	2	2	2026-09-02	4
5	1	2	2	2026-09-09	4
6	2	1	1	2026-09-01	2
7	4	1	1	2026-09-01	5
8	4	2	2	2026-09-02	5
9	5	1	1	2026-09-01	3
10	7	2	2	2026-09-02	4
11	8	3	1	2026-09-03	2
12	8	3	1	2026-09-10	5
13	8	1	1	2026-09-04	4

NULL

Прежде чем перейти к операциям с результатами запросов и соединениям таблиц, разберём, как SQL работает с отсутствующими значениями.

`NULL` – специальное значение-маркер, обозначающее отсутствие значения. Оно обрабатывается по особым правилам и может использоваться в столбцах разных типов.

Для проверки используют `IS NULL` и `IS NOT NULL`. Например, найдём группы без старосты:

```
SELECT group_id, code
  FROM groups
 WHERE admin_id IS NULL;
```

Результат:

```
 group_id | code
----------+-------
        3 | P3223
```

Трёхзначная логика

В PostgreSQL используется трёхзначная логика, поэтому работа с условиями немного отличается от привычной классической логики. В классической логике у высказывания два значения: истина (`TRUE`) и ложь (`FALSE`). В SQL появляется третье – неизвестно (`UNKNOWN`), которое представляется как `NULL` логического типа.

> Если значение нам неизвестно, то мы не можем определить, истинно сравнение или ложно. Например, неизвестное число может быть равно 5, а может отличаться от него. Поэтому результат `NULL = 5` – UNKNOWN (“неизвестно”).
>
> Отрицание не устраняет неопределённость: если мы не знаем, равно ли число 5, то мы также не знаем, отличается ли оно от 5. Поэтому `NULL <> 5` тоже даёт UNKNOWN.

Для операций с NULL можно представить следующую таблицу истинности:

Выражение	Результат	Пояснение
`TRUE AND NULL`	`NULL`	Для истинности AND нужны два истинных операнда, а второй неизвестен.
`FALSE AND NULL`	`FALSE`	Одного ложного операнда достаточно, чтобы AND был ложным.
`NULL AND NULL`	`NULL`	Оба операнда неизвестны, поэтому результат также неизвестен.
`TRUE OR NULL`	`TRUE`	Одного истинного операнда достаточно, чтобы OR был истинным.
`FALSE OR NULL`	`NULL`	Первый операнд ложный, поэтому результат зависит от неизвестного второго операнда.

Выражение	Результат	Пояснение
`NULL OR NULL`	`NULL`	Неизвестно, является ли истиной хотя бы одно из операндов.
`NOT NULL`	`NULL`	Отрицание неизвестного операнда тоже неизвестно.

> ⚠ Из-за описанных особенностей не используйте `= NULL` и `<> NULL` для проверки отсутствующего значения. Обычное сравнение с NULL даёт неизвестный результат, даже при сравнении NULL с самим собой.

Если нужно сравнить два значения, среди которых могут быть `NULL`, используют `IS DISTINCT FROM` и `IS NOT DISTINCT FROM`, которые для обычных значений работают как `<>` и `=`, но при сравнении двух `NULL` считают их неразличимыми:

```
SELECT NULL IS DISTINCT FROM NULL;     -- FALSE
SELECT NULL IS NOT DISTINCT FROM NULL; -- TRUE
SELECT 1 IS DISTINCT FROM NULL;        -- TRUE
```

Ловушка NOT IN

Попробуем найти студентов, которые не являются старостами:

```
SELECT student_id
  FROM students
 WHERE student_id NOT IN (
       SELECT admin_id FROM groups
 );
```

Запрос не вернёт строк. Подзапрос содержит `1`, `4` и `NULL`. Для старост условие ложно, а для остальных результат неизвестен, поскольку нельзя подтвердить отличие от неизвестного значения.

Для этой задачи можно использовать `NOT EXISTS`:

```
SELECT s.student_id
  FROM students AS s
 WHERE NOT EXISTS (
       SELECT 1
         FROM groups AS g
        WHERE g.admin_id = s.student_id
 )
 ORDER BY s.student_id;
```

Результат:

```
 student_id
------------
          2
          3
          5
          6
          7
          8
```

Множества и мультимножества

Определения

Результат SQL-запроса может содержать повторяющиеся строки. Чтобы понимать, когда повторения важны, а когда их нужно убрать, разберём простой пример.

Выведем идентификаторы студентов, у которых есть попытки сдачи экзамена по Базам данных (`exam_id = 1`):

```
SELECT student_id
  FROM attempts
  WHERE exam_id = 1
  ORDER BY student_id;
```

Ожидаемый вывод:

```
 student_id
------------
          1
          1
          1
          2
          4
          5
          8
```

Студент, у которого `student_id = 1`, встречается три раза, потому что у него три попытки сдачи.

Если нам нужен список студентов, достаточно оставить каждый идентификатор один раз. Если мы считаем попытки, повторения необходимо сохранить. Для описания этих двух случаев можно использовать понятия **множества** и **мультимножества**.

Множество (англ. *set*) – структура данных, которая представляет собой совокупность элементов без повторений.

Посмотрим на примере на Java:

```
Set<Integer> numbers = new HashSet<>();

numbers.add(1); // {1}
numbers.add(1); // {1}
numbers.add(2); // {1, 2}
numbers.add(3); // {1, 2, 3}
```

Мультимножество (англ. *multiset*) – структура данных, которая хранит совокупность элементов с учётом повторений.

```
List<Integer> numbers = new ArrayList<>();

numbers.add(1); // [1]
numbers.add(1); // [1, 1]
numbers.add(2); // [1, 1, 2]
numbers.add(3); // [1, 1, 2, 3]
numbers.add(3); // [1, 1, 2, 3, 3]
```

В Java `ArrayList` – список, а не мультимножество. Но в этом примере он демонстрирует именно поведение мультимножества: каждое добавление сохраняется, включая повторения. Отличие в том, что список также задаёт порядок элементов.

Большинство реализаций множества и мультимножества не гарантируют порядок хранения элементов. В SQL порядок строк результата также не гарантирован без `ORDER BY`.

В контексте SQL мы можем заметить, что обычный `SELECT` сохраняет повторяющиеся строки, поэтому его результат удобно рассматривать как мультимножество.

Такое поведение можно указать явно через `SELECT ALL`:

```
SELECT ALL student_id
  FROM attempts
 WHERE exam_id = 1
 ORDER BY student_id;
```

Результат будет таким же, как у предыдущего запроса. Слово `ALL` обычно опускают, поскольку это поведение по умолчанию.

Результат операции получения всех строк без повторений можно рассмотреть как множество. Для его выполнения добавим ключевое слово `DISTINCT`:

```
SELECT DISTINCT student_id
  FROM attempts
 WHERE exam_id = 1
 ORDER BY student_id;
```

Вывод:

```
 student_id
------------
          1
          2
          4
          5
          8
```

Операции над множествами

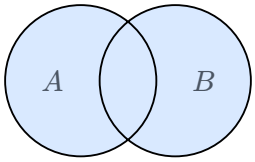
Рассмотрим два множества:

- A – идентификаторы студентов, у которых есть попытки сдачи экзамена по базам данных: `1, 2, 4, 5, 8`
- B – идентификаторы студентов, у которых есть попытки сдачи экзамена по математическому анализу: `1, 4, 7`

Для работы с результатами запросов в SQL есть операции объединения, пересечения и разности. На диаграммах ниже результат каждой операции выделен голубым.

Объединение: UNION

Объединение $A \cup B$ содержит элементы, которые входят хотя бы в одно из множеств.



Получим идентификаторы студентов, у которых есть попытки сдачи баз данных или математического анализа:

```
SELECT student_id
  FROM attempts
 WHERE exam_id = 1
UNION
SELECT student_id
  FROM attempts
 WHERE exam_id = 2
ORDER BY student_id;
```

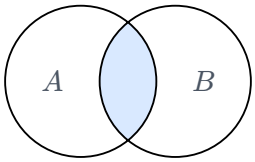
Результат:

```
 student_id
------------
          1
          2
          4
          5
          7
          8
```

Если студент сдавал оба экзамена или делал несколько попыток, его идентификатор всё равно появится один раз, поскольку `UNION` удаляет повторяющиеся строки.

Пересечение: INTERSECT

Пересечение $A \cap B$ содержит элементы, которые входят в оба множества.



Получим идентификаторы студентов, у которых есть попытки сдачи и баз данных, и математического анализа:

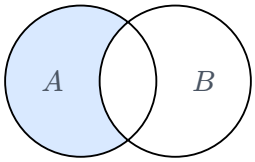
```
SELECT student_id
  FROM attempts
 WHERE exam_id = 1
INTERSECT
SELECT student_id
  FROM attempts
 WHERE exam_id = 2
ORDER BY student_id;
```

Результат:

```
 student_id
------------
          1
          4
```

Разность: EXCEPT

Разность $A \setminus B$ содержит элементы первого множества, которых нет во втором.



Получим идентификаторы студентов, у которых есть попытки сдачи баз данных, но нет попыток сдачи математического анализа:

```
SELECT student_id
  FROM attempts
 WHERE exam_id = 1
EXCEPT
SELECT student_id
  FROM attempts
 WHERE exam_id = 2
ORDER BY student_id;
```

Результат:

```
student_id
------------
          2
          5
          8
```

А что с повторениями?

По умолчанию UNION, INTERSECT и EXCEPT удаляют повторения. У каждой операции есть вариант с ALL, который учитывает количество одинаковых строк.

Например, идентификатор Ивана Иванова встречается три раза среди попыток по базам данных и два раза среди попыток по математическому анализу:

- UNION ALL оставит все пять вхождений.
- INTERSECT ALL оставит два вхождения (минимум).
- EXCEPT ALL оставит одно вхождение – разность, но не меньше нуля.

JOIN

Может показаться, что раз SQL уже умеет объединять, пересекать и вычитать множества, то операции соединений не нужны. На самом деле есть два принципиальных различия между JOIN и операциями с множествами.

- UNION, INTERSECT и EXCEPT – операции над множествами, из которых состоят результаты запросов. Они определяют, какие строки из двух результатов нужно сохранить.
- JOIN – операции над таблицами. Они решают задачи сопоставления строк таблиц по условию и формирования их сочетаний. При этом строки могут иметь разную структуру, а результат содержать данные с обеих сторон.

> Это одна из основных задач при работе с реляционной БД. Связанные сведения хранятся в разных таблицах, что позволяет уменьшать дублирование данных и поддерживать их согласованность. Соединения позволяют получать эти сведения вместе, в форме, необходимой для конкретного запроса.

Визуализация JOIN

JOIN часто визуализируют с помощью диаграмм Эйлера – Венна, которые на первый взгляд кажутся наглядными, хотя изначально предназначены для изображения множеств и отношений между ними, а не таблиц.

> Не используйте диаграммы Эйлера – Венна для объяснения JOIN, потому что они могут сформировать неправильное представление о работе соединений.

Есть несколько причин:

- Диаграммы показывают принадлежность элементов множествам, но не показывают условие сопоставления строк и количество получившихся сочетаний.
- Закрашенное пересечение кругов создаёт впечатление, что JOIN ищет одинаковые строки двух таблиц. На самом деле строки могут различаться и сопоставляться только по указанному условию.
- Некоторые соединения такая визуализация вообще не объясняет: например, CROSS JOIN формирует все возможные пары строк, а пересечение кругов этого не показывает.

Правильный выбор визуализации

Лучше всего визуализировать JOIN'ы именно так, как выглядят их результаты. В следующих разделах рассмотрим визуализации на примерах конкретных видов JOIN. На схемах покажем исходные строки и результат соединения, чтобы было видно, какие строки образуют пары и что происходит со строками без соответствий.

CROSS JOIN – перекрёстное соединение

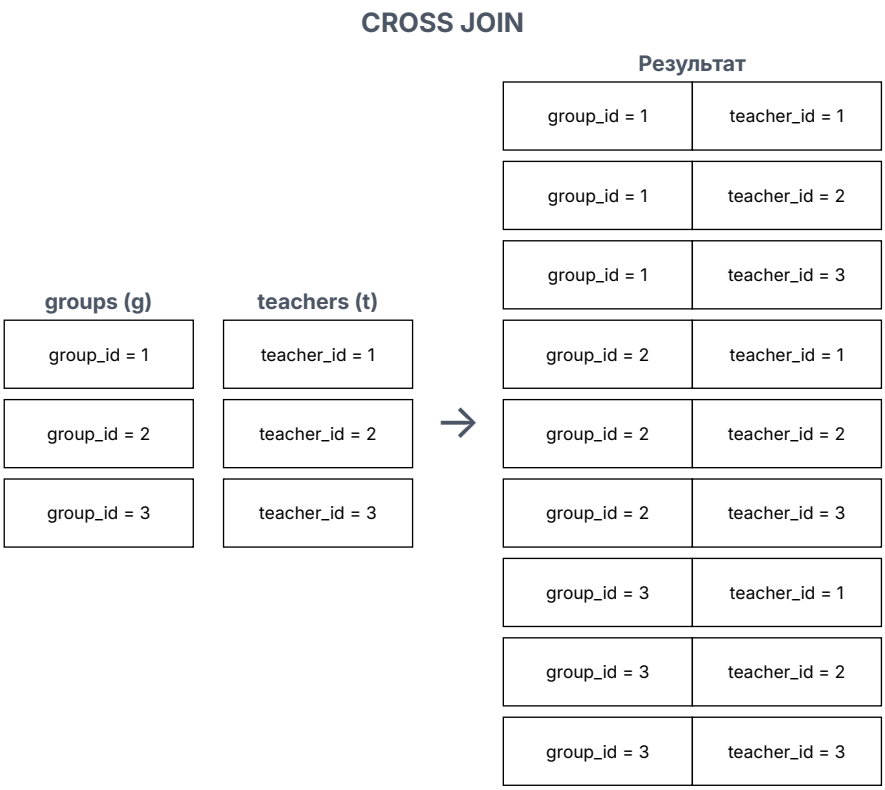
Формирует все возможные пары строк двух таблиц. Условие соединения не задаётся.

Получим все сочетания групп и преподавателей:

```
SELECT g.group_id, t.teacher_id
  FROM groups AS g
       CROSS JOIN teachers AS t
 ORDER BY g.group_id, t.teacher_id;
```

Результат:

```
 group_id | teacher_id
----------+------------
        1 |          1
        1 |          2
        1 |          3
        2 |          1
        2 |          2
        2 |          3
        3 |          1
        3 |          2
        3 |          3
```



INNER JOIN – внутреннее соединение

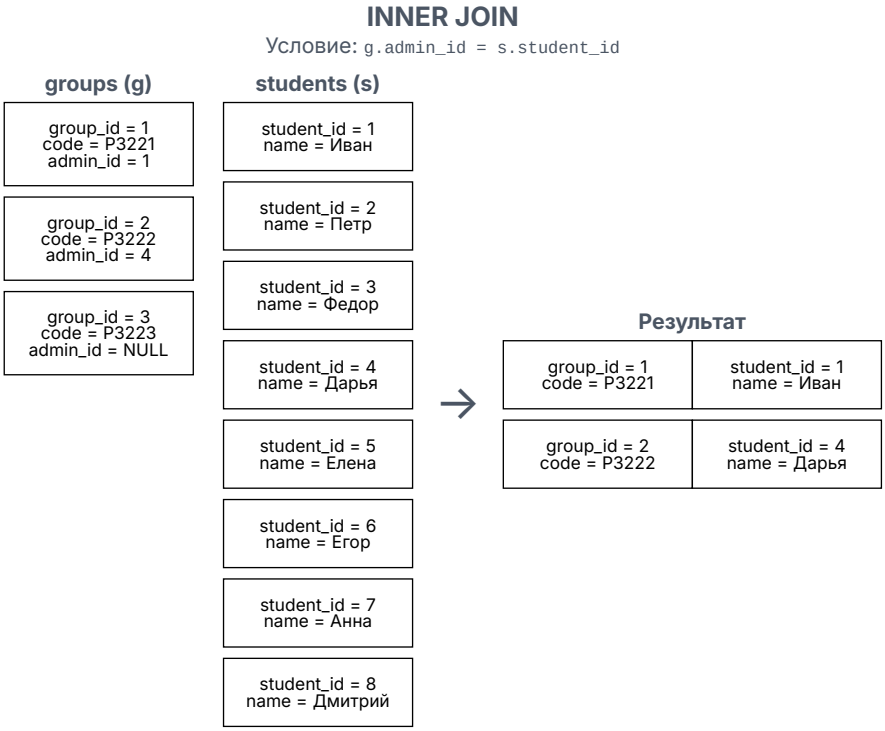
Возвращает только пары строк, удовлетворяющие условию соединения.

Получим группы, у которых назначен староста:

```
SELECT g.group_id, g.code, s.student_id, s.name
  FROM groups AS g
       INNER JOIN students AS s
       ON g.admin_id = s.student_id
 ORDER BY g.group_id;
```

Результат:

```
 group_id | code  | student_id | name
----------+-------+------------+-------
        1 | P3221 |          1 | Иван
        2 | P3222 |          4 | Дарья
```



Слово `INNER` можно опустить, обычный `JOIN` означает `INNER JOIN`.

LEFT JOIN – левое внешнее соединение

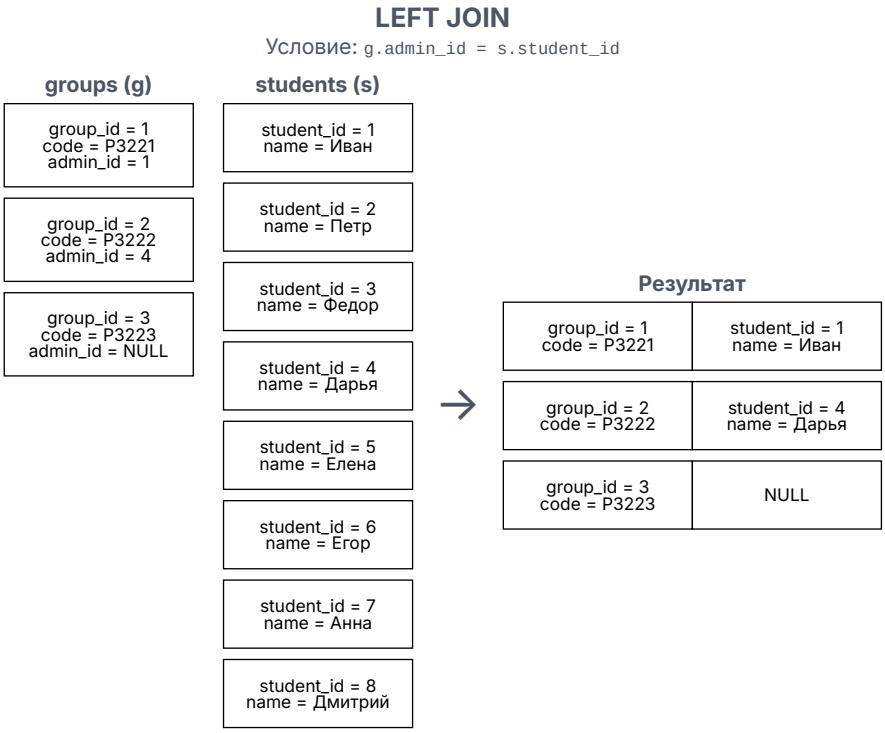
Возвращает подходящие пары и сохраняет строки левой таблицы без соответствий. Для них столбцы правой стороны заполняются `NULL`.

Получим все группы, включая группы без старост:

```
SELECT g.group_id, g.code, s.student_id, s.name
  FROM groups AS g
       LEFT JOIN students AS s
       ON g.admin_id = s.student_id
 ORDER BY g.group_id;
```

Результат:

```
 group_id | code  | student_id | name
----------+-------+------------+-------
        1 | P3221 |          1 | Иван
        2 | P3222 |          4 | Дарья
        3 | P3223 |            |
```



RIGHT JOIN – правое внешнее соединение

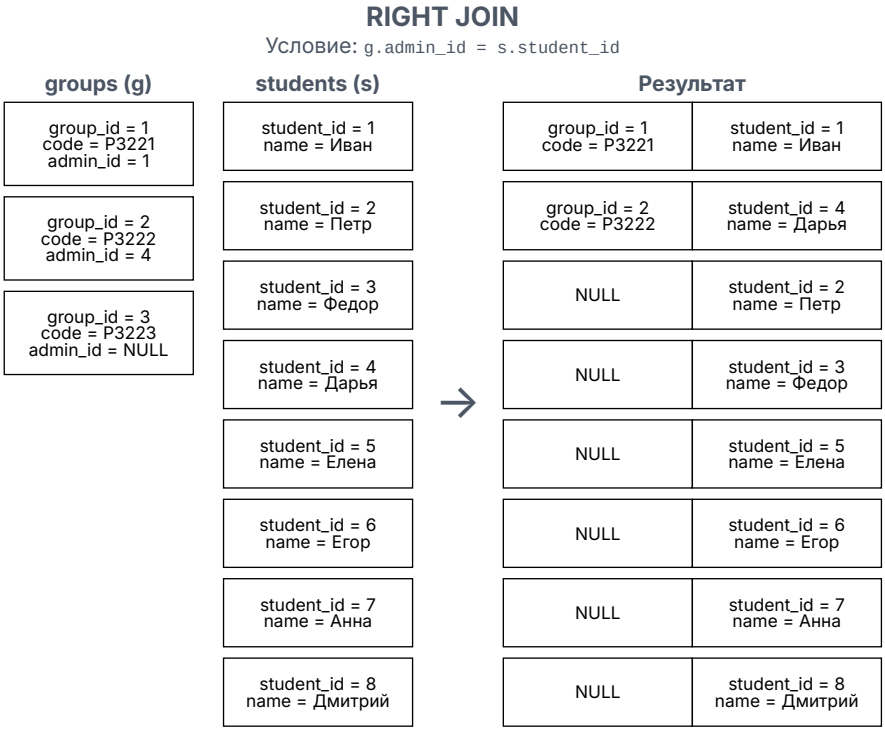
Возвращает подходящие пары и сохраняет строки правой таблицы без соответствий. Для них столбцы левой стороны заполняются `NULL`.

Получим всех студентов и сведения о группах, в которых они являются старостами:

```
SELECT g.group_id, g.code, s.student_id, s.name
  FROM groups AS g
       RIGHT JOIN students AS s
       ON g.admin_id = s.student_id
 ORDER BY g.group_id NULLS LAST, s.student_id;
```

Результат:

```
 group_id | code  | student_id |  name
----------+-------+------------+---------
        1 | P3221 |          1 | Иван
        2 | P3222 |          4 | Дарья
          |       |          2 | Петр
          |       |          3 | Федор
          |       |          5 | Елена
          |       |          6 | Егор
          |       |          7 | Анна
          |       |          8 | Дмитрий
```



> Если задачу можно выразить и через `LEFT JOIN`, и через `RIGHT JOIN`, для читаемости удобно придерживаться `LEFT JOIN`: слева располагается таблица, все строки которой нужно сохранить, а справа – дополнительные сведения. Такой порядок проще читать последовательно, слева направо.

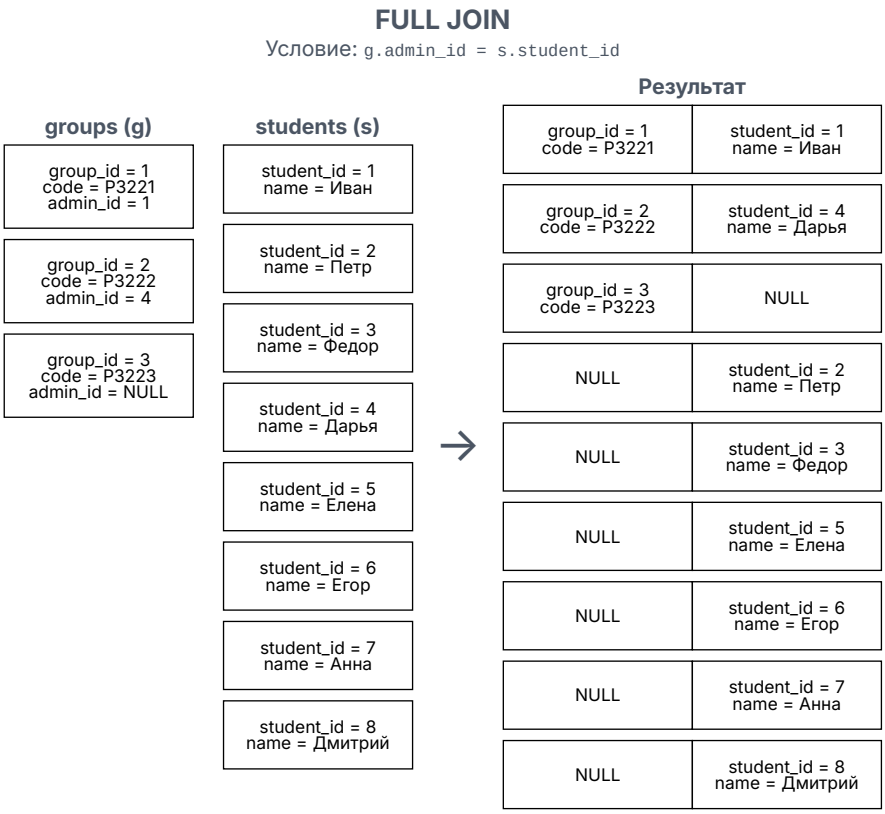
FULL JOIN – полное внешнее соединение

Возвращает подходящие пары и сохраняет строки без соответствий с обеих сторон. Столбцы отсутствующей стороны заполняются `NULL`.

```
SELECT g.group_id, g.code, s.student_id, s.name
  FROM groups AS g
       FULL JOIN students AS s
       ON g.admin_id = s.student_id
 ORDER BY g.group_id NULLS LAST, s.student_id;
```

Результат:

```
 group_id | code  | student_id |  name
----------+-------+------------+---------
        1 | P3221 |          1 | Иван
        2 | P3222 |          4 | Дарья
        3 | P3223 |            |
          |       |          2 | Петр
          |       |          3 | Федор
          |       |          5 | Елена
          |       |          6 | Егор
          |       |          7 | Анна
          |       |          8 | Дмитрий
```



> `LEFT JOIN`, `RIGHT JOIN` и `FULL JOIN` ещё часто называют `OUTER JOINS`, потому что такое соединение сохраняет не только подходящие пары, но и строки без соответствий с одной или обеих сторон. Слово необязательно, поскольку `LEFT`, `RIGHT` и `FULL` уже обозначают внешнее соединение. Например, `LEFT JOIN` и `LEFT OUTER JOIN` равнозначны.

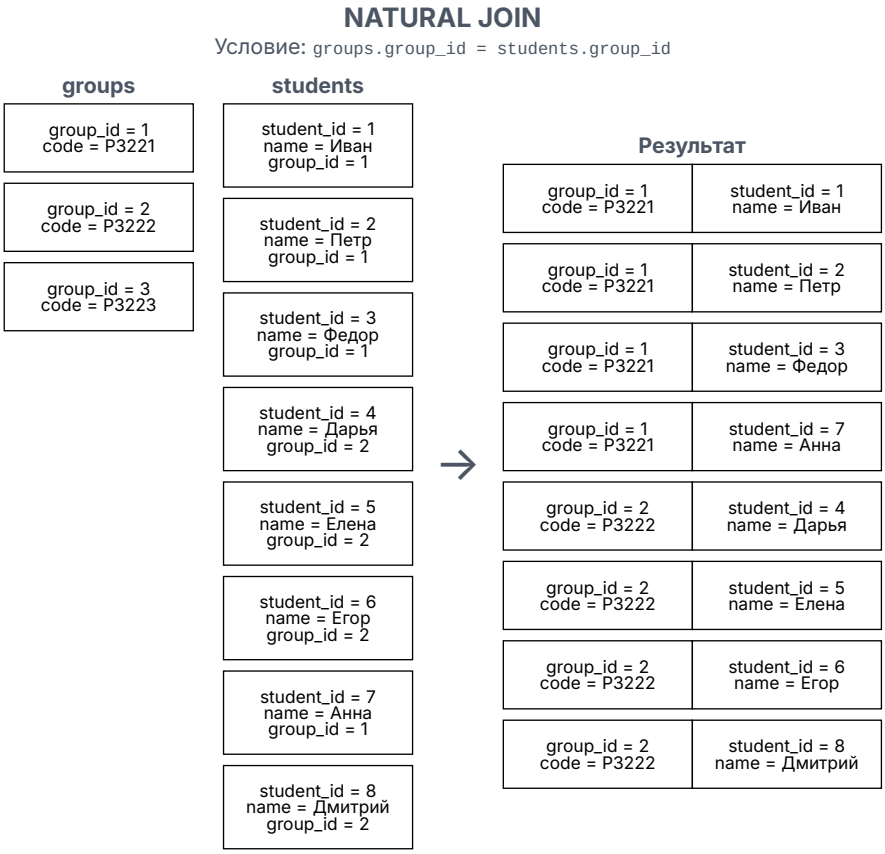
NATURAL JOIN – естественное соединение

Автоматически задаёт условие равенства по всем одноимённым столбцам двух таблиц. Общие столбцы представлены в результате по одному разу.

```
SELECT group_id, code, student_id, name
  FROM groups
       NATURAL JOIN students
 ORDER BY group_id, student_id;
```

Результат:

```
 group_id | code  | student_id |  name
----------+-------+------------+---------
        1 | P3221 |          1 | Иван
        1 | P3221 |          2 | Петр
        1 | P3221 |          3 | Федор
        1 | P3221 |          7 | Анна
        2 | P3222 |          4 | Дарья
        2 | P3222 |          5 | Елена
        2 | P3222 |          6 | Егор
        2 | P3222 |          8 | Дмитрий
```



Без уточнения `NATURAL JOIN` означает `NATURAL INNER JOIN`. Также допустимы `NATURAL LEFT JOIN`, `NATURAL RIGHT JOIN` и `NATURAL FULL JOIN`.

`NATURAL JOIN` учитывает все одноимённые столбцы, а не только ключи. Добавление нового одноимённого столбца может изменить условие соединения. Явные `ON` или `USING` позволяют избежать такой зависимости.

Обратная связь

Нашли опечатку или ошибку?

Есть идеи по улучшению материала?

Хотите предложить тему, которой можно будет дополнить доклад?

Пишите в личные сообщения или заведите issue в репозитории!

Источники

1. Материалы курса “Базы данных”. *Лекция 2: создание Базы Данных*.
2. Материалы курса “Базы данных”. *Лекция 4: Запросы и SQL*.
3. Eder, L. *Say NO to Venn Diagrams When Explaining JOINs*. Статья.
4. PostgreSQL 18. Combining Queries (UNION, INTERSECT, EXCEPT). Официальная документация.
5. PostgreSQL 18. Table Expressions. Виды соединений, условия ON и USING, NATURAL JOIN. Официальная документация.
6. PostgreSQL 18. Functions and Operators. Разделы Logical Operators и Comparison Functions and Operators: трёхзначная логика и сравнения с NULL.